CIS 4020 · DATA STRUCTURES

# Tables, Priority Queues, and Heaps

Notes on tables, priority queues, heaps, and heapsort, with Java and C/C++ examples

**A note before you begin.** A word before you read. These are my notes from years of teaching this unit, cleaned up so they are readable. While C/C++ was my first language, I am a bit rusty because I have taught Java for years now. For these notes, I have been using AI to help type these up, check for issues, and fill in explanations, which means a wrong or imprecise statement can slip in and survive longer than it should, and I do not fully trust the AI, so I am still editing. They are not meant to be a study guide, they are not gospel, and not everything here shows up in a lecture (and some lecture items are not here). If you find something that looks wrong, contradicts the book, or just does not behave the way I claim when you run it, tell me. You will be doing students and me a favor, and you will understand it better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. We are a two-language course, thus I will do my best to show everything in Java (CIS) and C/C++ (CS).

## 1. Where this unit fits

Lists, stacks, and queues are **position oriented**: you ask for an item by where it sits. The binary search tree was our first structure that is **value oriented** and fast at it: you ask for an item by its key, and the tree finds it without looking at most of the others.

This unit looks at two more value-oriented ADTs.

- A **table** lets you insert, delete, and retrieve any item by its search key.
- A **priority queue** lets you insert any item, but the only item you may remove is the one with the highest priority.

The priority queue asks for less than a table does, and that is exactly what lets it use a structure that is faster at its one job: the **heap**. Most of this unit is about heaps. We draw them, store them in an array and with references, turn them into a priority queue, and finally use them to sort.

## 2. The table ADT

A **table** (the book also calls it a **dictionary**, and Java and C++ call it a **map**) is a collection of items, each with a **search key**. Think of each item as a row and each field of the item as a column. The table is organized around the search key, so operations that use the key are fast, and anything else is a search through every row.

In Table 1 below, looking up New York is a search on the key, so the table can use whatever trick its implementation has (binary search, following a BST path) to find it quickly. Finding every city in Italy cannot use the key at all. The table has to visit all seven rows, and it finds Rome and Venice.

**Table 1:** A small table of cities with City as the search key. A search for the city New York uses the key. A search for every city in Italy does not, so it must look at every row.

| **City (search key)** | Country | Continent |
|---|---|---|
| **Athens** | Greece | Europe |
| **Cairo** | Egypt | Africa |
| **London** | United Kingdom | Europe |
| **New York** | United States | North America |
| **Rome** | Italy | Europe |
| **Tokyo** | Japan | Asia |
| **Venice** | Italy | Europe |

## Operations

The table operations follow the same wall idea as every ADT so far. The program asks for an operation, and how it happens is hidden.

- **Create** an empty table, and ask whether it **is empty** and what its **length** is.
- **Insert** a new item. Most tables do not allow two items with the same key.
- **Delete** the item with a given search key.
- **Retrieve** the item with a given search key.
- **Traverse** the items in sorted search key order, doing something with each one.

The search key is usually a field of the item. In our code, the item classes extend `KeyedItem`, the same class the binary search tree used, so every item can answer `getKey()`.

## Implementations

Every structure you already know can hold a table. The **linear** choices are an unsorted array, a sorted array, an unsorted linked list, and a sorted linked list. The **nonlinear** choice is a binary search tree.

Table 2, just after this list, gives the average cost of each operation. A few things in it are worth saying out loud.

- A **sorted array** can use binary search, so retrieval is fast. Insertion and deletion are slow because items must shift to keep the order.
- A **sorted linked list** keeps its order without shifting, but it cannot use binary search, because there is no way to jump to the middle of a linked list.
- A **binary search tree** is the only choice that is fast at all three of insertion, deletion, and retrieval, and its inorder traversal visits the items in sorted order for free. The catch is the shape of the tree, which is why the balanced trees from the previous unit matter so much here.

**Table 2:** Average time for each table operation. For the unsorted implementations, insertion is $O(1)$ only if the table does not check for a duplicate key first, and traversal in sorted order means sorting the items first. The binary search tree times assume a reasonably balanced tree. In the worst case (a tall, thin tree) insertion, deletion, and retrieval all become $O(n)$.

| Implementation | Insertion | Deletion | Retrieval | Sorted traversal |
|---|---|---|---|---|
| **Unsorted array** | $O(1)$ | $O(n)$ | $O(n)$ | $O(n \log n)$ |
| **Unsorted linked list** | $O(1)$ | $O(n)$ | $O(n)$ | $O(n \log n)$ |
| **Sorted array** | $O(n)$ | $O(n)$ | $O(\log n)$ | $O(n)$ |
| **Sorted linked list** | $O(n)$ | $O(n)$ | $O(n)$ | $O(n)$ |
| **Binary search tree** | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ | $O(n)$ |

## Choosing an implementation

There is no single best table. The right choice depends on which operations the program uses most and on what you know ahead of time.

- If the program mostly inserts and rarely retrieves (a log of events, for example), an **unsorted** array or list is hard to beat.
- If the table is built once and then searched many times (a dictionary that rarely changes), a **sorted array** gives fast binary search, and the slow insertions happen only once.
- If the program does a mix of everything, a **binary search tree**, ideally a balanced one, is the usual answer.
- If the maximum size is not known in advance, a linked list or a tree avoids the fixed size of an array.

The textbook also describes **hashing**, which stores items so that retrieval by key takes close to constant time on average, at the cost of giving up any sorted order. We will discuss hashing in a later set of lectures.

## A table built on a binary search tree

Once you have a working binary search tree, a table is mostly a wrapper around it. Each item in the tree is one row of the table, and each table operation passes straight through to a tree operation.

Sketch: a table that wraps the BST from the trees unit

```
class Table:
    bst = an empty binary search tree
    count = 0

    insert(item):      bst.add(item);        count = count + 1
    delete(key):       bst.remove(key);      count = count - 1
    retrieve(key):     return bst.search(key)
    traverse():        bst.inorder()          // sorted by key
    length():          return count
```

The table adds almost nothing of its own. That is the point: the hard work was done when the tree was built, and the table just gives the program a vocabulary (insert, delete, retrieve) that matches what it wants to do. We will see the same pattern again when a priority queue wraps a heap.

## 3. The priority queue ADT

A **priority queue** gives every item a **priority value**, which acts as its search key, and it serves items by priority rather than by arrival time. A few examples:

- An emergency room sees the most urgent patient next, not the one who arrived first.
- An operating system runs the most important waiting program next.
- A printer shared by an office may print short or urgent jobs ahead of long ones.
- Your own to-do list, if you are honest about it.

The operations are few.

- **Create** an empty priority queue, and ask whether it **is empty**.
- **Insert** an item with any priority.
- **Remove** the item with the **highest priority**, and return it.
- **Peek** at the item with the highest priority without removing it.

There is no way to retrieve or delete an arbitrary item, and no traversal. The ADT is built for exactly two operations, insert and remove, so those are the two we judge an implementation by.

### Three implementations you already know

- **Array-based (sorted).** Keep the items sorted by priority, with the highest priority at the **highest index**. Removing is fast: take the last item, and nothing shifts. Inserting is slow, because items shift to make room.
- **List-based (sorted).** Keep a sorted linked list with the highest priority at the **head**. Removing is fast: unlink the first node. Inserting is slow, because you walk the list to find the right spot.
- **Tree-based.** Keep a binary search tree. The highest priority is the **right-most node**, found by following right links from the root. The right-most node never has a right child, so removing it is always the easy one-child (or leaf) case of BST deletion.

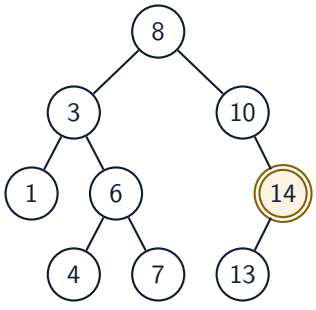



**Figure 1:** A binary search tree holding a priority queue. The lowest priority (1) is the left-most node, and the highest priority (14, highlighted) is the right-most node. Removing 14 replaces it with its only child, 13.

Each of these has a weakness. The sorted array and the sorted list are fast at one operation and slow at the other. The binary search tree is fast at both *if* it stays short, but nothing keeps it short. Worse, a priority queue always removes from the same end of the tree, so the right side shrinks and the tree drifts out of balance as it is used. A balanced search tree fixes the shape problem, but it is a lot of machinery for a queue that only ever needs its largest item. The heap is a simpler structure that is always short.

# 4. Heaps

## Complete binary trees, again

Recall from the trees unit that a binary tree of height $h$ is **complete** when every level above the last is full, and the nodes on the last level are filled in from the left with no gaps. A complete tree is as short as a binary tree with that many nodes can be.

## Definition

A **maxheap** is a complete binary tree that is either empty, or whose

- root holds a search key **greater than or equal to** the keys of its children, and
- root has heaps as its subtrees.

A **minheap** is the same, with **less than or equal to** in place of greater than or equal to. Like the BST definition, this one is recursive, so it holds at every node, not only the root.




**Figure 2:** Left: a maxheap. The root, 10 (highlighted), is the highest priority. Every key in the left subtree (9, 3, 2) and the right subtree (6, 5) is less than 10, and the same is true at 9 and at 6. Right: a minheap with the same six keys rearranged. The root, 2 (highlighted), is the smallest key, and keys increase as you go down any path.

A few observations from Figure 2:

- The **root is always the highest priority**: the largest key in a maxheap, the smallest key in a minheap. That is why a heap is a natural priority queue.
- **A heap is not a binary search tree.** The only rule is between a parent and its children. In the maxheap, 9 is the left child and 6 the right child, but they could just as well be swapped. Siblings have no order, and neither do cousins. 9 is larger than 6, but its children, 3 and 2, are both smaller than 6's child, 5.
- Because siblings are not ordered, a heap is **no good for searching**. To find an arbitrary key you may have to look at every node.

- The **last node** is the right-most node on the bottom level (5 in the maxheap, 6 in the minheap). Every insertion and deletion works at the last node, because that is the only place a node can be added or taken away while the tree stays complete.

## 5. Storing a heap in an array

A complete tree has no gaps, so its nodes can be stored in an array in breadth-first order: the root at index 0, then the next level left to right, and so on. No space is wasted, and no links are needed, because simple arithmetic finds a node's relatives.

**Table 3:** Index formulas for a heap stored from index 0. The divisions are integer divisions, which round down.

| **Relative of the node at index $i$** | Index | Exists when |
|---|---|---|
| **Parent** | $(i-1)/2$ | $i > 0$ |
| **Left child** | $2i+1$ | $2i+1 <$ size |
| **Right child** | $2i+2$ | $2i+2 <$ size |

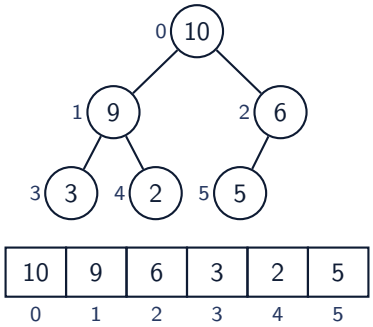



**Figure 3:** The maxheap from Figure 2 with each node's index written to its left, and the same heap stored in an array with the index under each box. Reading the tree level by level, left to right, gives the array: 10, 9, 6, 3, 2, 5.

Check the formulas against Figure 3. The node 9 is at index 1, so its children are at $2(1)+1=3$ and $2(1)+2=4$, which hold 3 and 2. The node 5 is at index 5, so its parent is at $(5-1)/2=2$, which holds 6. The node 6 at index 2 has a left child at index 5 but no right child, because index 6 is past the end of the heap.

Two more facts fall out of this layout.

- The **last node** is always at index size $-1$, and a new node always goes at index size.
- A node has at least one child exactly when its index is less than size/2. Every node from index size/2 on is a leaf.

Some books start the heap at index 1 instead of 0. Then the parent is $i/2$ and the children are $2i$ and $2i+1$. Both work. What does not work is mixing the two.

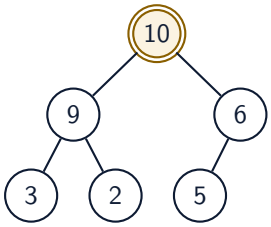
## 6. Removing from a heap

Removing always takes the root, because the root holds the highest priority. The hard part is closing the hole without breaking either half of the definition: the tree must stay complete, and

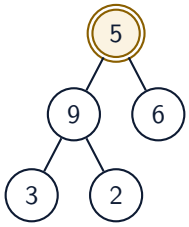
every parent must keep its priority above its children.

1. **Remove the root.** Save its item to return.
2. **Place the last node as the root.** Move the last node's item into the root and remove the last node. The tree is complete again, but the root is now probably in the wrong place. The book calls this tree a **semiheap**: both subtrees of the root are still heaps, and only the root is out of order.
3. **Trickle down.** While the item has a child with a higher priority, swap it with the **higher priority child** (the larger child in a maxheap). Stop when neither child outranks it, or when it reaches a leaf. If both children have the same key, swap with the left child.

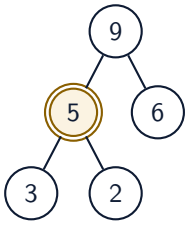
Swapping with the higher child matters. If 5 swapped with the smaller child in a maxheap, the smaller child would become the parent of the larger one, and the heap would be broken.




(a) Remove the root, 10.




(b) The last node, 5, becomes the root.




(c) 5 trickles down past 9 and stops.

**Figure 4:** Removing from the maxheap 10, 9, 6, 3, 2, 5. (a) The root, 10, is removed. (b) The last node's item, 5, moves into the root and the last node is gone. (c) 5 is smaller than its larger child, 9, so they swap. Now 5 is larger than both of its children, 3 and 2, so it stops. The highlighted node in each step is the item being removed or moved.

Table 4 continues removing until the heap is empty, showing each heap as an array. Draw a few of these as trees to check yourself.

**Table 4:** Removing every item from the maxheap 10, 9, 6, 3, 2, 5, one at a time. The items come out in decreasing order.

| **Operation** | Removed | Heap as an array afterward |
|---|---|---|
| **Start** | (none) | 10, 9, 6, 3, 2, 5 |
| **remove** | 10 | 9, 5, 6, 3, 2 |
| **remove** | 9 | 6, 5, 2, 3 |
| **remove** | 6 | 5, 3, 2 |
| **remove** | 5 | 3, 2 |
| **remove** | 3 | 2 |
| **remove** | 2 | (empty) |

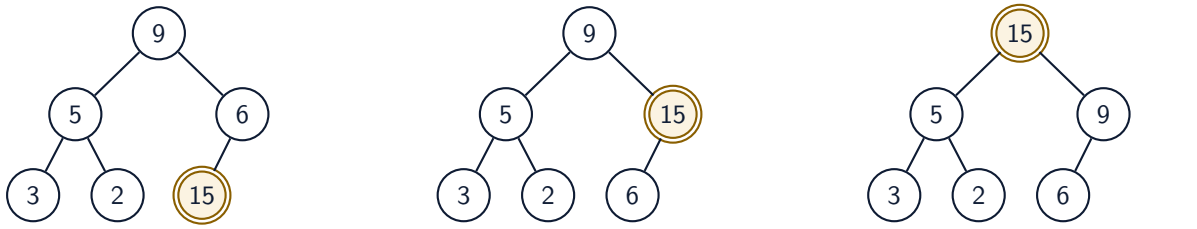
Notice that the items come out in sorted order, largest first. Hold on to that thought until the heapsort section.

## 7. Inserting into a heap

Insertion is the same idea run in the other direction.

1. **Place the new item as the last node.** That is the next open spot on the bottom level, or the first spot of a new level if the bottom level is full. The tree is still complete.
2. **Trickle up.** While the item has a higher priority than its parent, swap it with its parent. Stop when the parent outranks it (or ties it), or when it reaches the root.

Trickling up only ever compares with the parent, so there is no "which child" decision, which makes insertion the easier of the two.




(a) 15 is placed as the last node.

(b) 15 is larger than 6, so they swap.

(c) 15 is larger than 9, so they swap.

**Figure 5:** Inserting 15 into the maxheap 9, 5, 6, 3, 2. (a) 15, highlighted, goes in as the left child of 6, the next open spot. (b) 15 is larger than its parent, 6, so they swap. (c) 15 is larger than its new parent, 9, so they swap again. 15 is now the root, and it stops.

Table 5 continues the example with 10, 1, and 4.

**Table 5:** Inserting 15, 10, 1, and 4 into the maxheap 9, 5, 6, 3, 2. The middle column lists the parents the new item swapped with as it trickled up.

| **Operation** | Swapped with | Heap as an array afterward |
|---|---|---|
| **Start** | (none) | 9, 5, 6, 3, 2 |
| **insert 15** | 6, then 9 | 15, 5, 9, 3, 2, 6 |
| **insert 10** | 9 | 15, 5, 10, 3, 2, 6, 9 |
| **insert 1** | (none) | 15, 5, 10, 3, 2, 6, 9, 1 |
| **insert 4** | 3 | 15, 5, 10, 4, 2, 6, 9, 1, 3 |

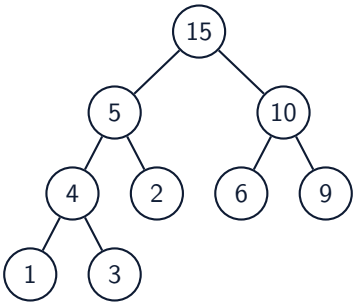



**Figure 6:** The maxheap after all four insertions: 15 at the root; 5 and 10 below it; 4, 2, 6, and 9 on the third level; and 1 and 3 as the children of 4.

### Swapping, or moving a hole

When we draw, we swap the moving item with its parent (or child) at every step. The code does something slightly smarter. It saves the moving item once, slides each parent down (or each child up) into the open spot, and writes the saved item only when it reaches its final place. The result is exactly the same heap, but each step costs one assignment instead of the three a swap needs. Keep this in mind when you trace the code: in the middle of a trickle, the moving item is in a local variable, not in the array.

## 8. Changing a key

Sometimes an item already in the heap needs a new priority. A patient's condition gets worse, or a waiting program has waited long enough that the system raises its priority. The heap can fix itself with the same two tools.

- If the new key gives the item a **higher** priority, the item may now outrank its parent, so it **trickles up**. Nothing below it can be out of place, since it only got better.
- If the new key gives the item a **lower** priority, one of its children may now outrank it, so it **trickles down**. Nothing above it can be out of place.

In a maxheap, a higher priority means a larger key, so raising a key trickles up. In a minheap, a higher priority means a *smaller* key, so lowering a key trickles up. It is easy to get this backward, so the code below asks "did the priority go up?" rather than "did the number go up?"

## 9. The array-based heap in code

The code is in the `heap` package (Java) and in `HeapArrayBased.h` (C++). Both versions throw a `HeapException` when something goes wrong, such as removing from an empty heap or inserting into a full one, the same way the BST threw a `BSTException`. The heap holds items that extend `KeyedItem`, and a constructor argument chooses a maxheap or a minheap.

### Maxheap or minheap: one comparison

The only difference between a maxheap and a minheap is which of two keys belongs higher in the tree. Every comparison in the code goes through one private method, so changing that one method changes the kind of heap.

Java: HeapArrayBased.java, the one comparison

```
// returns true when key1 belongs above key2:
// larger in a maxheap, smaller in a minheap
private boolean higherPriority(KT key1, KT key2) {
    int cmp = key1.compareTo(key2);
    if(type == HeapType.MAX) {
        return cmp > 0;
    } else {
        return cmp < 0;
    }
}
```

C++: HeapArrayBased.h, the one comparison

```
    // returns true when key1 belongs above key2:
    // larger in a maxheap, smaller in a minheap
    bool higherPriority(const KeyType& key1, const KeyType& key2) const {
        if (type == HeapType::MAX) {
            return key2 < key1;
        } else {
            return key1 < key2;
        }
    }
```

The C++ version only uses `<`, so a key type needs nothing more than a less than operator. Notice that `higherPriority` returns `false` for equal keys. An item never moves past an equal one, which saves useless swaps.

## Insert and trickle up

Java: HeapArrayBased.java

```
    public void insert(T item) {
        if(item == null) {
            throw new HeapException("Heap Exception: Null Item");
        }
        if(size == maxSize) {
            throw new HeapException("Heap Exception: Heap Full");
        }
        heap[size] = item; // step 1: place the new item as the last node
        trickleUp(size);   // step 2: trickle up
        size++;
    }

    private void trickleUp(int index) {
        int parent = (index - 1) / 2;
        T bottom = heap[index]; // the item that is moving up

        // while not at the root and the new item outranks its parent
        while(index > 0 && higherPriority(bottom.getKey(), heap[parent].getKey())) {
            heap[index] = heap[parent]; // move the parent down
            index = parent;
            parent = (parent - 1) / 2;
        }
        heap[index] = bottom;
    }
```

The C++ version keeps its items in a `std::vector` that is never allowed to grow past `maxSize`, so it behaves like an array. The vector means an item type does not need a default constructor, the same reason the C++ list used `std::optional`.

C++: HeapArrayBased.h

```
    void insert(const ItemType& item) override {
        if (size() == maxSize) {
```

```
            throw HeapException("Heap Exception: Heap Full");
        }
        heap.push_back(item);    // step 1: place the new item as the last node
        trickleUp(size() - 1);   // step 2: trickle up
    }

    void trickleUp(int index) {
        int parent = (index - 1) / 2;
        ItemType bottom = heap[index];  // the item that is moving up

        // while not at the root and the new item outranks its parent
        while (index > 0 && higherPriority(bottom.getKey(), heap[parent].getKey())) {
            heap[index] = heap[parent];  // move the parent down
            index = parent;
            parent = (parent - 1) / 2;
        }
        heap[index] = bottom;
    }
```

## Remove and trickle down

Java: HeapArrayBased.java

```
    public T remove() {
        if(size == 0) {
            throw new HeapException("Heap Exception: Empty Heap");
        }
        T root = heap[0];     // step 1: remove the root

        size--;
        heap[0] = heap[size]; // step 2: place the last node as the root
        heap[size] = null;
        if(size > 0) {
            trickleDown(0);   // step 3: trickle down
        }
        return root;
    }

    private void trickleDown(int index) {
        int higherChild;
        T top = heap[index]; // the item that is moving down

        while(index < size / 2) { // while the node has at least one child
            int leftChild = 2 * index + 1;
            int rightChild = leftChild + 1;
            // find the child with the higher priority
            // (the larger child in a maxheap, the smaller child in a minheap)
            if(rightChild < size
                    && higherPriority(heap[rightChild].getKey(), heap[leftChild].getKey
    ())) {
                higherChild = rightChild; // rightChild exists and wins
            } else {
                higherChild = leftChild;
```

```
            }
            // stop when the child does not outrank the item moving down
            if(!higherPriority(heap[higherChild].getKey(), top.getKey())) {
                break;
            }
            heap[index] = heap[higherChild]; // move the child up
            index = higherChild;             // go down
        }
        heap[index] = top;
    }
```

Two details in `trickleDown` catch people.

- The loop test `index < size / 2` means "this node has at least one child," using the fact from the array section.
- A node can have a left child and no right child (at most one node in the heap is like this). The test `rightChild < size` has to come first, so the code never reads past the end of the heap.

C++: HeapArrayBased.h

```
    ItemType remove() override {
        if (heap.empty()) {
            throw HeapException("Heap Exception: Empty Heap");
        }
        ItemType root = heap[0];  // step 1: remove the root

        heap[0] = heap.back();    // step 2: place the last node as the root
        heap.pop_back();
        if (!heap.empty()) {
            trickleDown(0);       // step 3: trickle down
        }
        return root;
    }
```

The C++ `trickleDown` is line for line the same as the Java one, with `heap.size()` in place of `size`.

## Change

Java: HeapArrayBased.java

```
    public void change(int index, KT newKey) {
        if(index < 0 || index >= size) {
            throw new HeapException("Heap Exception: Invalid Index");
        }
        if(newKey == null) {
            throw new HeapException("Heap Exception: Null Key");
        }
        KT oldKey = heap[index].getKey(); // remember old
        heap[index].setKey(newKey);       // change to new
        if(higherPriority(newKey, oldKey)) {
            trickleUp(index);   // priority went up, so trickle it up
        } else {
```

```
            trickleDown(index); // priority went down (or stayed), so trickle it down
        }
    }
```

The index check is `index >= size`, not `index > size`. Index `size` is the first empty slot, not an item. The C++ `change` is the same apart from syntax.

### Trying it out

The driver `Heaping` (Java) or `Heaping.cpp` (C++) asks which kind of heap and which storage to use, inserts 70, 40, 50, 20, 60, 100, 80, 30, 10, and 90, and then lets you insert, delete, peek, show, and change. It prints each heap as an array and as a tree.

Output: Heaping, array-based maxheap, after the ten insertions

```
Heap: 100 90 80 30 60 50 70 20 10 40
------------------------------------------------------------
                                100
                90                              80
        30              60              50              70
    20      10      40
------------------------------------------------------------
```

Run it a second time and choose a minheap. The same ten insertions put 10 at the root.

## 10. A reference-based heap

A heap can also be built from linked nodes, like the binary search tree. Each node holds an item and links to its children, plus one more link, to its **parent**, so an item can trickle up without starting over at the root.

Java: Node.java (heap package)

```
class Node<T> {
    T item;
    Node<T> left;
    Node<T> right;
    Node<T> parent;

    public Node(T item) {
        this.item = item;
        this.left = null;
        this.right = null;
        this.parent = null;
    }
}
```

The trickles barely change. Instead of moving between indexes, the code moves between nodes: `current.parent` on the way up, `current.left` or `current.right` on the way down, and it moves *items* between nodes rather than moving the nodes themselves.

## Finding the last node

The real problem is the one thing the array made free. Insertion needs the spot just past the last node, and removal needs the last node itself. In an array those are indexes `size` and `size - 1`. With links, there is no index to jump to.

The trick is to number the nodes anyway, exactly the way the array does, and then use the array formulas to walk there. Node $i$ has parent $(i-1)/2$. Its left child has an odd number $(2i+1)$ and its right child an even number $(2i+2)$. So to reach node $i$: first reach its parent, then go left if $i$ is odd and right if $i$ is even. That is a short recursive method, and it is the same recursion pattern as every tree method so far.

Java: HeapReferenceBased.java

```java
// returns node number index by walking down from the root
// note: assumes 0 <= index < size
private Node<T> nodeAt(int index) {
    if(index == 0) {
        return root;
    }
    Node<T> parent = nodeAt((index - 1) / 2); // first find the parent
    if(index % 2 == 1) {
        return parent.left;  // odd numbers are left children
    } else {
        return parent.right; // even numbers are right children
    }
}
```

C++: HeapReferenceBased.h

```cpp
// returns node number index by walking down from the root
// note: assumes 0 <= index < count
Node* nodeAt(int index) const {
    if (index == 0) {
        return root;
    }
    Node* parent = nodeAt((index - 1) / 2);  // first find the parent
    if (index % 2 == 1) {
        return parent->left;   // odd numbers are left children
    } else {
        return parent->right;  // even numbers are right children
    }
}
```

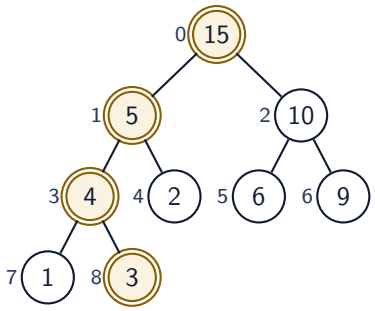



**Figure 7:** The heap from Figure 6 with each node's number written to its left. The highlighted path leads to node 8, the last node. Node 8 is even, so it is the right child of node $(8-1)/2 = 3$. Node 3 is odd, so it is the left child of node 1. Node 1 is odd, so it is the left child of node 0, the root. Reading back down: from 15 go left to 5, left to 4, then right to 3.

With `nodeAt`, both operations follow the same steps as before.

Java: HeapReferenceBased.java

```
    public void insert(T item) {
        if(item == null) {
            throw new HeapException("Heap Exception: Null Item");
        }
        Node<T> newNode = new Node<T>(item);

        // step 1: place the new node as the last node (node number size)
        if(root == null) {
            root = newNode;
        } else {
            Node<T> parent = nodeAt((size - 1) / 2);
            newNode.parent = parent;
            if(size % 2 == 1) {
                parent.left = newNode;
            } else {
                parent.right = newNode;
            }
        }
        size++;
        trickleUp(newNode); // step 2: trickle up
    }

    public T remove() {
        if(root == null) {
            throw new HeapException("Heap Exception: Empty Heap");
        }
        T rootItem = root.item; // step 1: remove the root item

        if(size == 1) {
            root = null;
        } else {
            // step 2: place the last node's item in the root
            Node<T> last = nodeAt(size - 1);
            root.item = last.item;
            // then unlink the last node from its parent
            if(last.parent.right == last) {
```

```
                last.parent.right = null;
            } else {
                last.parent.left = null;
            }
            last.parent = null;
            trickleDown(root); // step 3: trickle down
        }
        size--;
        return rootItem;
    }
```

In Figure 7, the next insertion becomes node 9. Its parent is node $(9-1)/2 = 4$ (the 2), and 9 is odd, so the new node is the left child of 2. A removal copies 3 (node 8) into the root, unlinks node 8 from 4, and trickles 3 down.

The C++ version is the same, with one addition you should expect by now: the removed last node has to be freed with `delete`, and the destructor frees every node that is left. Copying a heap is turned off for the same reason as the BST, so two heaps never share the same nodes.

C++: HeapReferenceBased.h, the part of remove that differs

```
            // then unlink the last node from its parent and free it
            if (last->parent->right == last) {
                last->parent->right = nullptr;
            } else {
                last->parent->left = nullptr;
            }
            delete last;
            trickleDown(root);  // step 3: trickle down
```

**Another way to see it.** Add 1 to a node's number and write the result in binary. After the leading 1, each bit is one step from the root: 0 means left, 1 means right. For node 8, $8 + 1 = 9 = 1001$ in binary, and the bits after the leading 1 are 0, 0, 1: left, left, right. This is the same walk `nodeAt` takes, just read from the top down instead of built from the bottom up.

## Array-based or reference-based?

**Table 6:** Comparing the two heap implementations.

| Question | Array-based | Reference-based |
|---|---|---|
| **Size limit** | Fixed when created | Only the memory available |
| **Extra memory per item** | None | Three links (left, right, parent) |
| **Finding the last node** | $O(1)$, index size $-1$ | $O(\log n)$, walk from the root |
| **Insert and remove** | $O(\log n)$ | $O(\log n)$ |
| **Moving to a parent or child** | Arithmetic | Follow a link |

Because a heap is always complete, the array never has holes, and that makes the array-based heap the usual choice in practice. The priority queues built into both languages, `PriorityQueue` in Java

and `std::priority_queue` in C++, keep their heap in an array that grows when it fills up. The reference-based heap is still worth building once, because it shows that the heap is really a tree and that the array is only a clever way to store one.

## 11. How fast is a heap?

A heap is complete, so it is always as short as possible. Counting levels the way the book does, a complete tree with $n$ nodes has height $\lceil \log_2(n+1) \rceil$, so a path from the root to a leaf takes at most $\lceil \log_2(n+1) \rceil - 1$ steps. A trickle up or trickle down moves one level per step, so it can never take more steps than that.

**Table 7:** Running time of priority queue operations for each implementation. For the binary search tree, the first time is for a reasonably balanced tree and the second is the worst case. The sorted array keeps the highest priority at the end, and the sorted linked list keeps it at the head. The heap times are worst case, because a heap can never become unbalanced.

| **Implementation** | Insert | Remove highest | Peek |
|---|---|---|---|
| **Sorted array** | $O(n)$ | $O(1)$ | $O(1)$ |
| **Sorted linked list** | $O(n)$ | $O(1)$ | $O(1)$ |
| **Unsorted array** | $O(1)$ | $O(n)$ | $O(n)$ |
| **Binary search tree** | $O(\log n)$ or $O(n)$ | $O(\log n)$ or $O(n)$ | $O(\log n)$ or $O(n)$ |
| **Heap** | $O(\log n)$ | $O(\log n)$ | $O(1)$ |

To get a feel for $O(\log n)$: a heap with a million items is only 20 levels deep, so an insertion or a removal makes at most 19 moves. The sorted array in the same situation might shift half a million items for a single insertion.

The heap gives up something for this speed. It cannot search for an arbitrary key faster than $O(n)$, and it cannot list its items in order without taking them all out. That is a fine trade for a priority queue, which never asks for either one. It would be a bad trade for a table.

### A heap or a balanced search tree?

A 2-3 tree (or any balanced search tree) also inserts and removes in $O(\log n)$ time, so it is fair to ask why a priority queue would use a heap at all. For the two operations a priority queue needs, the heap wins in several ways.

- **Peek is $O(1)$.** The highest priority is always at the root. In a 2-3 tree, the largest item is in the right-most leaf, so finding it means walking down the tree.
- **No links are needed.** A heap fits in an array with no gaps. A 2-3 tree node has room for two values and three child links, and many of those slots sit empty.
- **The code is much simpler.** Heap insert and remove are each one short loop. A 2-3 tree needs splitting, redistributing, and merging.
- **Building from a batch is faster.** If all the items are known up front, the heapsort build phase makes a heap in $O(n)$ time. Building a 2-3 tree takes $O(n \log n)$.

The heap is faster because it keeps less order. A balanced search tree keeps every item in sorted order, so it can search for any key, delete any item, list the items in order, and reach both the

smallest and the largest item in $O(\log n)$ time. A heap can do none of these quickly. Choose a heap when the program only ever needs the highest priority item. Choose a balanced search tree when it also needs to search, remove arbitrary items, or see the items in order.

This is why the priority queues built into Java and C++ both use an array-based heap, while their sorted maps use a balanced tree.

## 12. A priority queue built on a heap

The heap does all the work, so the priority queue class is a thin wall around it, just like the table that wraps a binary search tree. The program sees only the priority queue operations. The heap's extra operations, `change` and `displayHeap`, are not part of the priority queue ADT, so they stay hidden.

This code is a separate project from the heap project, so the heap code can be studied on its own and the priority queue can be studied as a user of it.

Java: PriorityQueue.java (pq package, abridged)

```
public class PriorityQueue<T extends KeyedItem<KT>, KT extends Comparable<? super KT>>
    {

    private HeapInterface<T, KT> heap;

    // no size limit, so it uses the reference-based heap
    // HeapType.MAX serves the largest key first, HeapType.MIN the smallest
    public PriorityQueue(HeapType type) {
        heap = new HeapReferenceBased<T, KT>(type);
    }

    // a fixed size, so it uses the array-based heap
    public PriorityQueue(int maxSize, HeapType type) {
        heap = new HeapArrayBased<T, KT>(maxSize, type);
    }

    public boolean isEmpty() {
        return heap.isEmpty();
    }

    // removes and returns the item with the highest priority
    public T remove() {
        try {
            return heap.remove();
        } catch(HeapException e) {
            throw new PriorityQueueException("Priority Queue Exception: Empty Queue");
        }
    }
    // insert, peek, size, and removeAll follow the same pattern
}
```

Two design choices are worth pointing out.

- The field has the **interface** type `HeapInterface`, so the priority queue works the same with

either heap, and the constructor decides which one to use. A program that uses the priority queue cannot tell the difference.

- The priority queue **translates** a `HeapException` into a `PriorityQueueException`. A program using a priority queue should not have to know that a heap exists, so it should not have to catch a heap's exception either.

The C++ `PriorityQueue.h` holds its heap in a `std::unique_ptr<HeapInterface<ItemType, KeyType>>`, so the heap is freed automatically when the priority queue goes away, and it translates exceptions the same way.

The driver `PQApp` uses two queues. A to-do list uses a maxheap, so the largest priority number comes out first. An emergency room uses a minheap, because triage level 1 is the most urgent.

Output: PQApp

```
To-do list, largest priority first:
Next up: Finish the project (priority 9)
  Finish the project (priority 9)
  Study for the quiz (priority 7)
  Answer email (priority 5)
  Read Chapter 12 (priority 3)
  Clean the desk (priority 1)

Triage, level 1 is the most urgent:
Priority Queue Exception: Queue Full
Patients waiting: 4
  Heart attack (priority 1)
  Broken arm (priority 2)
  Sprained ankle (priority 4)
  Common cold (priority 5)
Priority Queue Exception: Empty Queue
```

The triage queue holds at most four patients, so the fifth insertion fails, and the last line shows what happens when a program removes from an empty queue.

## 13. Heapsort

Table 4 showed that removing every item from a maxheap produces the items largest first. That is already a sorting algorithm: insert everything into a heap, then remove everything. Heapsort does exactly this, with two improvements. It works inside the array being sorted, with no second array or separate heap object, and it builds the starting heap faster than $n$ separate insertions would.

### Phase 1: build a heap

Look at the unsorted array as a complete tree. Every leaf is already a (one-node) heap. Starting at the last node that has a child, index $n/2-1$, and working back to the root, trickle each item down. Only this first index is calculated. The loop then counts down by one (index 2, then 1, then 0 in Table 8), trickling down each item in turn. Nodes past index $n/2-1$ are leaves, so they are skipped. When an item trickles down, both of its subtrees are already heaps, so each step turns a semiheap into a heap. The book calls this trickle down `rebuild` (or `heapRebuild`), and our heapsort uses that name.

**Table 8:** Phase 1 on the array 6, 3, 5, 9, 2, 10. Each row trickles down one item, starting at index $6/2-1=2$ and working back to index 0.

| Step | Array afterward |
|---|---|
| **Start** | 6, 3, 5, 9, 2, 10 |
| **rebuild index 2 (the 5)** | 6, 3, 10, 9, 2, 5 |
| **rebuild index 1 (the 3)** | 6, 9, 10, 3, 2, 5 |
| **rebuild index 0 (the 6)** | 10, 9, 6, 3, 2, 5 |

The result is the same maxheap we drew in Figure 2.

## Phase 2: move the root to the end, and repeat

The root is the largest item, and it belongs at the end of the array. So swap the root with the last item of the heap, shrink the heap by one (the end of the array is now the sorted part, which only grows), and rebuild from the root. Repeat until the heap has one item left.

**Table 9:** Phase 2 on the heap from Table 8. A vertical bar separates the heap part (left) from the sorted part (right).

| Step | Array afterward |
|---|---|
| **Heap built** | 10, 9, 6, 3, 2, 5 |
| **Swap root with index 5, trickle down** | 9, 5, 6, 3, 2 \| 10 |
| **Swap root with index 4, trickle down** | 6, 5, 2, 3 \| 9, 10 |
| **Swap root with index 3, trickle down** | 5, 3, 2 \| 6, 9, 10 |
| **Swap root with index 2, trickle down** | 3, 2 \| 5, 6, 9, 10 |
| **Swap root with index 1, trickle down** | 2 \| 3, 5, 6, 9, 10 |

Each row is a swap followed by a trickle down. Here is the "index 4" row in full.

1. **Start of the step:** 9, 5, 6, 3, 2 | 10. The heap is indexes 0 to 4, and the 10 is already sorted.
2. **Swap the root with the last item in the heap:** the 9 goes to index 4 and the 2 goes to the root, giving 2, 5, 6, 3 | 9, 10. The 9 is now in its final place, so the heap shrinks to indexes 0 to 3.
3. **Trickle the 2 down within those four slots:** its children are 5 and 6, and the larger is 6, so they swap, giving 6, 5, 2, 3 | 9, 10. The 2 is now at index 2, whose children would be at indexes 5 and 6. Both are outside the heap, so it stops.

Compare the heap parts in Table 9 with Table 4. They are identical. Phase 2 *is* heap removal; the only difference is that each removed item lands in the space the heap just gave up.

## The code

Java: HeapSort.java (sort package, without the tracing output)

```
    public static <T extends Comparable<? super T>> void heapSort(T[] items) {
        int n = items.length;
```

```
        // phase 1: build a maxheap
        // start at the last node that has a child and work back to the root
        for(int index = n / 2 - 1; index >= 0; index--) {
            rebuild(items, index, n);
        }

        // phase 2: move the largest item to the end, then rebuild the rest
        for(int last = n - 1; last > 0; last--) {
            // swap the root with the last item in the heap
            T temp = items[0];
            items[0] = items[last];
            items[last] = temp;
            // the heap is now items[0] to items[last - 1]
            rebuild(items, 0, last);
        }
    }
```

`rebuild(items, index, size)` is `trickleDown` from the heap class, written for a plain array of `Comparable` items and a heap that ends at `size`. The `size` argument is what keeps phase 2 from trickling into the sorted part.

C++: HeapSort.h (without the tracing output)

```
template <typename T>
void heapSort(std::vector<T>& items) {
    int n = static_cast<int>(items.size());

    // phase 1: build a maxheap
    // start at the last node that has a child and work back to the root
    for (int index = n / 2 - 1; index >= 0; index--) {
        rebuild(items, index, n);
    }

    // phase 2: move the largest item to the end, then rebuild the rest
    for (int last = n - 1; last > 0; last--) {
        // swap the root with the last item in the heap
        std::swap(items[0], items[last]);
        // the heap is now items[0] to items[last - 1]
        rebuild(items, 0, last);
    }
}
```

The driver `HeapSorting` sorts 6, 3, 5, 9, 2, 10 and prints every step, which reproduces Tables 8 and 9, and then sorts a list of names to show that heapsort works on any type that can be compared.

A maxheap sorts into increasing order, because the largest item is moved to the back first. A minheap would sort into decreasing order.

### How heapsort compares

- Phase 2 does $n - 1$ rebuilds of $O(\log n)$ each, so heapsort is $O(n \log n)$. Phase 1 turns out to be only $O(n)$, because most items start near the bottom and have only a short way to trickle.
- Heapsort is $O(n \log n)$ in the **worst case**, not just on average, unlike quicksort, which can fall to $O(n^2)$.
- Heapsort sorts **in place**. It needs only a few extra variables, unlike mergesort, which needs a second array as large as the first.
- Heapsort is **not stable**. Two items with equal keys can come out in the opposite order from how they went in.
- In practice, a well written quicksort is usually faster on average, because it works on neighboring array slots, which suits the computer's memory caches. Heapsort jumps between distant indexes.

## 14. Common mistakes

- **Treating a heap like a binary search tree.** There is no left-smaller, right-larger rule. Only parents and children are ordered, so you cannot search a heap by going left or right.
- **Putting a new item anywhere but the last position.** A new item always goes in the next open spot on the bottom level, left to right. Anywhere else, and the tree stops being complete.
- **Trickling down toward the wrong child.** In a maxheap, always swap with the *larger* child. Swapping with the left child out of habit, or with the smaller child, breaks the heap one level down.
- **Forgetting the node with only a left child.** Check that the right child exists before comparing with it.
- **Mixing the two index conventions.** With the root at index 0, the children of $i$ are $2i + 1$ and $2i + 2$. With the root at index 1, they are $2i$ and $2i + 1$. Pick one.
- **Getting the direction backward after a change.** A higher priority trickles up, and a lower priority trickles down. In a minheap, a higher priority is a smaller key.
- **Leaving the last node behind.** After moving the last item to the root, the last node must actually go away: the size shrinks, and in the reference-based heap the parent's link is set to `null` (and, in C++, the node is deleted).
- **Letting heapsort rebuild into the sorted part.** Each rebuild in phase 2 must stop at the current end of the heap, not the end of the array.

A good habit when you draw a heap by hand: after every insertion or deletion, check both halves of the definition. Is the tree complete? Does every parent outrank its children?

## 15. Further reading

- HackerRank, *Data Structures: Heaps* (video, also linked from the slides). `https://www.youtube.com/watch?v=t0Cq6tVNRBA`
- VisuAlgo, *Binary Heap (Priority Queue)*: an interactive maxheap that shows insert, remove, change key, and heapsort as a tree and as an array.

`https://visualgo.net/en/heap`

- David Galles, *Heap Visualization* (University of San Francisco): an interactive minheap. `https://www.cs.usfca.edu/~galles/visualization/Heap.html`

The last two links were found with help from an AI system, not from my own prior use. They look relevant, but tell me if one is broken or off-topic.